

Efficient Partition Trees — Calculation Notes

matousek-partition-tree / docs

Contents

1	Efficient Partition Trees: Calculation Notes	2
1.1	Global Conventions	2
1.2	Section 2: Cuttings	2
1.3	Randomized Cutting Construction in Section 2	3
1.4	Section 3: Partition Theorem	4
1.5	Lemma 3.2: Fixed Test Set Q	4
1.5.1	Iterative Construction	4
1.5.2	Weight Growth	6
1.6	Lemma 3.3: Test Set Lemma	8
1.6.1	Construction of Q	8
1.6.2	Why Q Controls Every h	9
1.7	Proof of Theorem 3.1	10
1.8	Tightness Argument in Theorem 3.1	11
1.9	Lemma 3.4: Constant r Construction Time	11
1.10	Corollary 3.5: Simple Algorithmic Partition	12
1.10.1	Composition Calculation	12
1.11	Section 4: Epsilon-Approximations	13
1.12	Observation 4.1	14
1.13	Theorem 4.2	15
1.14	Lemma 4.3: Approximation Implies Cutting	15
1.15	Proposition 4.4	15
1.16	Lemma 4.5: Auxiliary Range Counting	16
1.17	Lemma 4.6: Fast Partition for Small r	17
1.18	Theorem 4.7	18
1.18.1	Part 1	18
1.18.2	Part 2	18
1.18.3	Part 3	19
1.19	Theorem 4.8: Improved Cutting Computation	19
1.20	Theorem 5.1: Basic Partition Tree	20
1.21	Corollary 5.2: Space/Query Tradeoff	22
1.22	Section 6: Reducing Query Time	22
1.23	Why Theorem 5.1 Has a Log Factor	22
1.24	Lemma 6.2: k-Tuple Containment Structure	23
1.25	Lemma 6.3: Reporting Segments Crossing a Hyperplane	24
1.26	Theorem 6.1 Final Calculation	25
1.27	Section 7: Dynamization	26
1.27.1	Deletions	27
1.27.2	Insertions	27

Efficient Partition Trees: Calculation Notes

Source: Jiří Matoušek, “Efficient Partition Trees”, Discrete & Computational Geometry 8:315-334, 1992.

Coverage: calculation-focused extraction. This records the quantitative dependencies, recurrence bounds, parameter choices, and proof algebra. It does not reproduce historical prose or bibliography details except where they affect the bounds.

Global Conventions

- Dimension is fixed: d is treated as a constant.
- Hidden constants may depend on d and on fixed epsilon-style constants.
- P is an n -point set in Euclidean d -space.
- H is usually a set of n hyperplanes.
- s is the desired class size in a simplicial partition.
- r is usually:

$$r = \frac{n}{s}$$

- A simplicial partition is a collection:

$$\Pi = \{(P_1, \Delta_1), \dots, (P_m, \Delta_m)\}$$

where the sets P_i partition P , and each relatively open simplex Δ_i contains P_i .

- The class-size condition is:

$$\max_i |P_i| < 2 \min_i |P_i|$$

In the main theorem this becomes:

$$s \leq |P_i| < 2s$$

- A hyperplane h crosses a simplex Δ if:

$$h \cap \Delta \neq \emptyset$$

and:

$$\Delta \not\subset h$$

- The crossing number of h relative to Π is the number of simplices Δ_i crossed by h .
- The crossing number of Π is the maximum over all hyperplanes h .

Section 2: Cuttings

For a set H of n hyperplanes and a cutting Ξ , define:

$$H_\Delta = \{h \in H : h \text{ intersects the interior of } \Delta\}$$

A cutting Ξ is an epsilon-cutting if:

$$|H_\Delta| \leq \varepsilon n$$

for every simplex $\Delta \in \Xi$.

Weighted version:

$$w(X) = \sum_{h \in X} w(h)$$

A weighted epsilon-cutting satisfies:

$$w(H_\Delta) \leq \varepsilon w(H)$$

Theorem 2.1 gives:

there exists a $(1/r)$ -cutting of size $O(r^d)$

and it can be computed in:

$$O(nr^{d-1})$$

Weighted cuttings have the same asymptotic size and running-time bounds.

Randomized Cutting Construction in Section 2

The paper sketches the standard two-level cutting algorithm:

1. Sample r hyperplanes from H .
2. Build and triangulate the arrangement of the sample.
3. For each sample simplex Δ , compute:

$$H_\Delta = \{h \in H : h \text{ intersects } \Delta\}$$

4. Define:

$$t_\Delta = \frac{|H_\Delta|r}{n}$$

5. Sample:

$$Ct_\Delta \log t_\Delta$$

hyperplanes from H_Δ , add the boundary hyperplanes of Δ , triangulate inside Δ , and combine all resulting simplices.

The resulting cutting has expected optimal size:

$$O(r^d)$$

and expected construction time:

$$O(nr^{d-1})$$

Section 3: Partition Theorem

Main theorem:

Given P , n , s , and:

$$r = \frac{n}{s}$$

there exists a simplicial partition Π with:

$$s \leq |P_i| < 2s$$

and crossing number:

$$O(r^{1-1/d})$$

This is tight in the worst case.

Lemma 3.2: Fixed Test Set Q

Input:

- P , n , s .
- Q , a finite set of hyperplanes.

Output:

A simplicial partition Π with:

$$s \leq |P_i| < 2s$$

such that every q in Q has crossing number:

$$O(r^{1-1/d} + \log |Q|)$$

1.5.1 Iterative Construction

After i rounds:

- Already selected point classes:

$$P_1, \dots, P_i$$

- Remaining points:

$$P'_i = P \setminus (P_1 \cup \dots \cup P_i)$$

- Remaining count:

$$n_i = |P'_i|$$

If:

$$n_i < 2s$$

then set the last class to all remaining points and terminate.

For q in Q , let:

$$x_i(q) = \#\{\Delta_1, \dots, \Delta_i \text{ crossed by } q\}$$

Define weights:

$$w_i(q) = 2^{x_i(q)}$$

Thus each time q crosses a newly selected simplex, its weight doubles.

Let total weight be:

$$w_i(Q) = \sum_{q \in Q} w_i(q)$$

Choose t_i as large as possible such that there exists a weighted $(1/t_i)$ -cutting for (Q, w_i) whose simplices have at most:

$$\frac{n_i}{s}$$

faces of all dimensions in total.

By Theorem 2.1, since a cutting with parameter t_i has $O(t_i^d)$ faces, t_i can satisfy:

$$t_i > c \left(\frac{n_i}{s} \right)^{1/d}$$

for some positive constant c .

Since the cutting has at most:

$$\frac{n_i}{s}$$

faces and there are n_i remaining points, the pigeonhole principle gives a relatively open face containing at least s remaining points.

Choose such a face as:

$$\Delta_{i+1}$$

Pick exactly s points inside it:

$$P_{i+1} \subset P'_i \cap \Delta_{i+1}$$

Then continue.

1.5.2 Weight Growth

Let:

$$Q_{i+1} = \{q \in Q : q \text{ crosses } \Delta_{i+1}\}$$

For q in Q_{i+1} , the weight doubles. For all other q , the weight stays the same.

Therefore:

$$w_{i+1}(Q) = w_i(Q) - w_i(Q_{i+1}) + 2w_i(Q_{i+1})$$

so:

$$w_{i+1}(Q) = w_i(Q) + w_i(Q_{i+1})$$

and:

$$w_{i+1}(Q) = w_i(Q) \left(1 + \frac{w_i(Q_{i+1})}{w_i(Q)}\right)$$

Because Δ_{i+1} is a face of a weighted $(1/t_i)$ -cutting:

$$w_i(Q_{i+1}) \leq \frac{w_i(Q)}{t_i}$$

For full-dimensional faces this is exactly the cutting guarantee. For lower-dimensional faces, a hyperplane crossing the face also intersects the interior of an adjacent full-dimensional simplex, so the same bound holds up to a constant absorbed in c .

Thus:

$$w_{i+1}(Q) \leq w_i(Q) \left(1 + \frac{1}{t_i}\right)$$

Using:

$$t_i > c \left(\frac{n_i}{s}\right)^{1/d}$$

we get:

$$\frac{1}{t_i} < \frac{1}{c} \left(\frac{s}{n_i}\right)^{1/d}$$

In nonterminal rounds exactly s points are removed, so:

$$n_i = n - is$$

and:

$$r = \frac{n}{s}$$

The number of nonterminal rounds is:

$$m = \lfloor r \rfloor$$

Hence:

$$n_i = s(r - i)$$

so:

$$\left(\frac{s}{n_i}\right)^{1/d} = \frac{1}{(r - i)^{1/d}}$$

Iterating the weight inequality:

$$w_m(Q) \leq |Q| \prod_{i=0}^{m-1} \left(1 + \frac{1}{c(r - i)^{1/d}}\right)$$

Taking logarithms and using:

$$\ln(1 + x) \leq x$$

gives:

$$\log w_m(Q) \leq \log |Q| + \frac{1}{c} \sum_{i=0}^{m-1} \frac{1}{(r - i)^{1/d}}$$

Rewrite the sum by $j = r - i$:

$$\sum_{i=0}^{m-1} \frac{1}{(r - i)^{1/d}} \leq \sum_{j=1}^r \frac{1}{j^{1/d}}$$

Bound by integral:

$$\sum_{j=1}^r j^{-1/d} = O(r^{1-1/d})$$

Therefore:

$$\log w_m(Q) = O(\log |Q| + r^{1-1/d})$$

If a hyperplane q has final crossing number x , then:

$$w_m(q) = 2^x$$

Since:

$$w_m(q) \leq w_m(Q)$$

we get:

$$x \leq \log_2 w_m(Q) = O(\log |Q| + r^{1-1/d})$$

This proves Lemma 3.2.

Lemma 3.3: Test Set Lemma

Purpose: reduce all possible hyperplanes to a finite test set Q .

Input:

- P, n .
- Parameter r .

Output:

A set Q of at most r hyperplanes such that if a simplicial partition has all classes of size at least s , and if:

$$x_0 = \max_{q \in Q} \text{cr}_\Pi(q)$$

then every hyperplane h has crossing number:

$$\text{cr}_\Pi(h) \leq (d+1)x_0 + O\left(\frac{n}{sr^{1/d}}\right)$$

1.6.1 Construction of Q

Dualize P :

$$H = \mathcal{D}(P)$$

This is a set of n hyperplanes.

Choose a cutting for H with cutting parameter:

$$t = \Omega(r^{1/d})$$

The cutting has $O(t^d)$ vertices. Choose the constant so that the total number of vertices is at most r .

Let V be the set of vertices of the cutting. Define:

$$Q = \mathcal{D}(V)$$

Then:

$$|Q| \leq r$$

1.6.2 Why Q Controls Every h

Take an arbitrary hyperplane h .

Its dual point:

$$\mathcal{D}(h)$$

lies in some cutting simplex A .

Let G be the set of vertices of A . A d -dimensional simplex has $d + 1$ vertices.

The dual hyperplanes:

$$\mathcal{D}(G)$$

belong to Q .

Each of these $d + 1$ test hyperplanes crosses at most x_0 simplices of Π , so simplices crossed by at least one of them are at most:

$$(d + 1)x_0$$

Now consider simplices crossed by h but by no hyperplane in $\mathcal{D}(G)$.

Such simplices must be completely contained in the zone of h in the arrangement of $\mathcal{D}(G)$.

By duality, any point p of P lying in the interior of that zone has dual hyperplane $\mathcal{D}(p)$ intersecting the interior of A .

Because A is a $(1/t)$ -cutting cell and:

$$t = \Omega(r^{1/d})$$

the number of dual hyperplanes crossing A is:

$$O\left(\frac{n}{r^{1/d}}\right)$$

Thus the number of original points in those exceptional simplices is:

$$O\left(\frac{n}{r^{1/d}}\right)$$

Each partition class has at least s points, so the number of exceptional simplices is:

$$O\left(\frac{n}{sr^{1/d}}\right)$$

Therefore:

$$cr_{\Pi}(h) \leq (d + 1)x_0 + O\left(\frac{n}{sr^{1/d}}\right)$$

This is the Test Set Lemma.

Proof of Theorem 3.1

Use Lemma 3.3 to get:

$$|Q| \leq r$$

Then use Lemma 3.2 on this Q.

For every q in Q:

$$\text{cr}_{\Pi}(q) = O(r^{1-1/d} + \log |Q|)$$

Since:

$$|Q| \leq r$$

we have:

$$\log |Q| \leq \log r$$

and for fixed d at the relevant scale:

$$\log r = O(r^{1-1/d})$$

so:

$$x_0 = O(r^{1-1/d})$$

Now apply Lemma 3.3:

$$\text{cr}_{\Pi}(h) \leq (d+1)O(r^{1-1/d}) + O\left(\frac{n}{sr^{1/d}}\right)$$

Since:

$$r = \frac{n}{s}$$

we have:

$$\frac{n}{sr^{1/d}} = \frac{r}{r^{1/d}} = r^{1-1/d}$$

Therefore:

$$\text{cr}_{\Pi}(h) = O(r^{1-1/d})$$

for every hyperplane h.

Tightness Argument in Theorem 3.1

Choose a set Q of hyperplanes in general position whose arrangement has at least:

$$\left\lceil \frac{n}{s-1} \right\rceil$$

distinct cells.

It suffices to take:

$$O(r^{1/d})$$

hyperplanes, because an arrangement of M hyperplanes in fixed dimension d has:

$$\Theta(M^d)$$

cells.

Place n points into clusters of at most $s - 1$ points each, with clusters in distinct cells of the arrangement of Q .

In any simplicial partition with classes of size at least s , no class can be contained in one such cluster. Therefore every simplex must cross at least one hyperplane of Q .

There are about:

$$\frac{n}{s} = r$$

simplices/classes.

The total number of crossings over all hyperplanes in Q is at least:

$$\Omega(r)$$

Since:

$$|Q| = O(r^{1/d})$$

the average crossing number of a hyperplane in Q is:

$$\Omega\left(\frac{r}{r^{1/d}}\right) = \Omega(r^{1-1/d})$$

Thus the upper bound is asymptotically tight.

Lemma 3.4: Constant r Construction Time

If r is bounded by a constant:

- Test Set Lemma needs a cutting with constant parameter, computable in:

$$O(n)$$

- Q has constant size.
- Lemma 3.2 makes only constant many steps and handles constant many simplices.

The only issue is large weights:

$$w_i(q) = 2^{x_i(q)}$$

For weighted cutting computation, it is enough to know relative weights within a constant factor, because the weighted algorithm replaces h by approximately:

$$m(h) = \left\lfloor \frac{|H|w(h)}{w(H)} \right\rfloor + 1$$

copies.

Thus ordinary integer arithmetic with constant relative accuracy is enough.

Conclusion:

$$O(n)$$

time for constant r .

Corollary 3.5: Simple Algorithmic Partition

Goal:

Construct a partition with class size s and crossing number:

$$O(r^{1-1/d+\delta})$$

in:

$$O(n \log r)$$

for fixed positive δ .

1.10.1 Composition Calculation

First build a partition with:

$$r_1 = \frac{n}{s_1}$$

and crossing number:

$$Cr_1^{1-1/d}$$

Then refine each class with:

$$r_2 = \frac{2s_1}{s_2}$$

and crossing number:

$$Cr_2^{1-1/d}$$

The combined crossing number is bounded by a constant times:

$$C^2 r_1^{1-1/d} r_2^{1-1/d} = O((r_1 r_2)^{1-1/d})$$

The paper writes the resulting multiplicative loss as:

$$2C^2 r^{1-1/d}$$

where:

$$r = \frac{n}{s_2}$$

Repeat with a fixed constant reduction factor r_0 .

Number of iterations:

$$D = \frac{\log r}{\log r_0}$$

Each iteration loses a constant factor, so the total loss is:

$$(2C)^D = r^{\log(2C)/\log r_0}$$

Choose r_0 large enough so:

$$\frac{\log(2C)}{\log r_0} < \delta$$

Then final crossing number:

$$O(r^{1-1/d+\delta})$$

Total time over levels:

$$O(n \log r)$$

Section 4: Epsilon-Approximations

An epsilon-approximation A for P with respect to simplices satisfies:

$$\left| \frac{|A \cap \sigma|}{|A|} - \frac{|P \cap \sigma|}{|P|} \right| < \varepsilon$$

Weighted form:

$$\left| \frac{w(A \cap \sigma)}{w(A)} - \frac{|P \cap \sigma|}{|P|} \right| < \varepsilon$$

Observation 4.1

Given a simplicial partition:

$$\Pi = \{(P_1, \Delta_1), \dots, (P_m, \Delta_m)\}$$

with:

$$|P_i| \leq s$$

and crossing number x .

Pick one representative point:

$$a_i \in P_i$$

with weight:

$$w(a_i) = |P_i|$$

Let:

$$A = \{a_1, \dots, a_m\}$$

For a query simplex σ , the representative count differs from the true count only for partition simplices crossed by the boundary of σ .

A d -dimensional simplex has $d + 1$ bounding hyperplanes. Each boundary hyperplane crosses at most x partition simplices.

So at most:

$$(d + 1)x$$

classes can contribute error.

Each class has at most s points, so absolute error is at most:

$$(d + 1)xs$$

Relative error is:

$$\varepsilon = \frac{(d + 1)xs}{n}$$

Thus:

$$(A, w)$$

is an epsilon-approximation with:

$$\varepsilon = \frac{(d + 1)xs}{n}$$

Theorem 4.2

Using Corollary 3.5 and Observation 4.1, compute a $(1/t)$ -approximation of size:

$$O(t^{d+\delta})$$

in:

$$O(n \log t)$$

The parameter is chosen so that the partition crossing error satisfies:

$$\frac{(d+1)xs}{n} \leq \frac{1}{t}$$

with:

$$x = O(r^{1-1/d+\delta})$$

and:

$$r = \frac{n}{s}$$

This yields representative set size:

$$O(r) = O(t^{d+\delta})$$

after absorbing constant-dimension exponent slack.

Lemma 4.3: Approximation Implies Cutting

For hyperplanes H , let A be an epsilon-approximation for H with respect to segments.

If Ξ is an epsilon-prime cutting for weighted A , then Ξ is a:

$$d(\varepsilon + \varepsilon')$$

cutting for H .

The factor d comes from reducing simplex-cell crossing control to segment-edge crossing control in fixed dimension.

Proposition 4.4

Goal: compute a $(1/r)$ -cutting of size:

$$O(r^d)$$

for n hyperplanes.

Step 1: compute a $(1/(2dr))$ -approximation of H of size:

$$O(r^{d+\delta})$$

in time:

$$O(n \log r)$$

Step 2: compute a $(1/(2dr))$ -cutting for the approximation.

The approximation size is:

$$N_A = O(r^{d+\delta})$$

Using Theorem 2.1:

$$O(N_A r^{d-1}) = O(r^{d+\delta} r^{d-1}) = O(r^{2d+\delta-1})$$

Total:

$$O(n \log r + r^{2d+\delta-1})$$

If:

$$r < n^{1/(2d+\delta-1)}$$

then:

$$r^{2d+\delta-1} < n$$

so total time is:

$$O(n \log r)$$

Lemma 4.5: Auxiliary Range Counting

Given:

- fixed constant C ,
- point set P of size n ,
- parameter r satisfying:

$$r < n^\alpha$$

for sufficiently small $\alpha = \alpha(C)$.

Build in:

$$O(n \log r)$$

a structure answering simplex counting queries in:

$$O\left(\frac{n}{r^C}\right)$$

and supporting deletions in:

$$O(1)$$

per deletion.

Construction:

Choose t as a sufficiently large power of r .

Build a simplicial partition with at most t classes and crossing number:

$$O(t^{1-1/d+\delta})$$

Each class has size about:

$$\frac{n}{t}$$

For a query simplex, directly inspect classes crossed by the boundary, and add stored counts for fully contained classes.

Query time:

$$O\left(t + \frac{n}{t}t^{1-1/d+\delta}\right)$$

Simplify:

$$O\left(t + nt^{-1/d+\delta}\right)$$

Choose t as a sufficiently large power of r and choose α small enough so:

$$O\left(t + nt^{-1/d+\delta}\right) \leq O\left(\frac{n}{r^\alpha}\right)$$

Lemma 4.6: Fast Partition for Small r

If:

$$r < n^\alpha$$

for sufficiently small fixed $\alpha > 0$, then the exact Partition Theorem partition can be constructed in:

$$O(n \log r)$$

Key costs:

1. Test-set cutting with:

$$t = \Omega(r^{1/d})$$

computed using Proposition 4.4 in:

$$O(n \log r)$$

2. In Lemma 3.2, selecting a face containing enough remaining points requires counting points in cutting faces across rounds.

There are:

$$O(r^2)$$

range counting queries total in the argument as stated in the paper.

Using Lemma 4.5, all dynamic range-counting and reporting operations are covered within:

$$O(n \log r)$$

for sufficiently small α .

Theorem 4.7

Three algorithmic partition bounds.

1.18.1 Part 1

If:

$$s \geq n^\delta$$

then:

$$r = \frac{n}{s} \leq n^{1-\delta}$$

The construction can be iterated a constant number of times using Lemma 4.6.

Result:

- class sizes:

$$s \leq |P_i| < 2s$$

- crossing number:

$$O(r^{1-1/d})$$

- time:

$$O(n \log r)$$

1.18.2 Part 2

For arbitrary s , iterate the partition construction at most:

$$O(\log \log n)$$

times.

Each iteration loses a constant factor in crossing number.

Total multiplicative loss:

$$2^{O(\log \log n)} = (\log n)^{O(1)}$$

Result:

$$O(r^{1-1/d}(\log r)^{O(1)})$$

crossing number in:

$$O(n \log r)$$

time.

1.18.3 Part 3

For arbitrary s , exact optimal crossing:

$$O(r^{1-1/d})$$

can be constructed in:

$$O(n^{1+\delta})$$

Idea:

- Iterate fast construction until residual class sizes are small:

$$O(n^{\delta'})$$

- On small classes, use the polynomial-time exact construction from Theorem 3.1.
- Choose δ' small enough so total time is:

$$O(n^{1+\delta})$$

Theorem 4.8: Improved Cutting Computation

For:

$$r \leq n^{1-\delta}$$

a $(1/r)$ -cutting of size:

$$O(r^d)$$

can be computed in:

$$O(n \log r + \sqrt{nr}^{d-1/2})$$

Calculation:

Choose intermediate parameter:

$$r_1 = \left\lceil \frac{r^{1+1/(2(d-1))}}{n^{1/(2(d-1))}} \right\rceil$$

Compute a $(1/r_1)$ -cutting of H and conflict lists H_Δ .

Then for each cell Δ , compute a (r_1/r) -cutting for:

$$H_\Delta \cup B_\Delta$$

where B_Δ are boundary hyperplanes of Δ .

Balancing the top-level and refinement costs yields:

$$O(n \log r + \sqrt{nr}^{d-1/2})$$

Theorem 5.1: Basic Partition Tree

Build a recursive partition tree.

At a node v with m points, choose:

$$s = \lfloor m^{1/d} \rfloor$$

Then:

$$r = \frac{m}{s} = m^{1-1/d}$$

Class size after one level:

$$m^{1/d}$$

Thus node sizes decrease by repeated d -th roots:

$$n, \quad n^{1/d}, \quad n^{1/d^2}, \quad \dots$$

Tree depth:

$$O(\log \log n)$$

At a node with m points, the crossing number of the partition is:

$$O(r^{1-1/d}(\log r)^{O(1)})$$

Using:

$$r = m^{1-1/d}$$

this is:

$$O\left(m^{(1-1/d)^2}(\log m)^{O(1)}\right)$$

But the recurrence in the paper is stated abstractly as:

$$T(n) \leq O(r) + O(r^{1-1/d})T(s)$$

with:

$$r = \frac{n}{s}$$

and:

$$s = n^{1/d}$$

So:

$$r = n^{1-1/d}$$

and:

$$T(n) \leq O(n^{1-1/d}) + O(n^{(1-1/d)^2})T(n^{1/d})$$

The solution is:

$$T(n) = O(n^{1-1/d}(\log n)^{O(1)})$$

Space:

An m-point node stores:

$$O(r) = O(m^{1-1/d})$$

simplices and cumulative weights.

Summing over levels gives:

$$O(n)$$

space.

Preprocessing:

An m-point node costs:

$$O(m \log m)$$

The node sizes decrease double-exponentially with depth, so total preprocessing is:

$$O(n \log n)$$

Corollary 5.2: Space/Query Tradeoff

Combine partition tree with a fast simplex range searching structure.

For storage parameter m satisfying:

$$n < m < n^d$$

preprocessing and space:

$$O(m^{1+\delta})$$

query time:

$$O\left(\frac{n}{m^{1/d}} \log^{d+1} n\right)$$

The paper writes the same tradeoff in asymptotic form with fixed-dimension constants.

Section 6: Reducing Query Time

Theorem 6.1 gives linear space but improved query factors, with worse preprocessing.

General semigroup:

$$O(n^{1-1/d}(\log \log n)^{O(1)})$$

with:

$$O(n^{1+\delta})$$

preprocessing and:

$$O(n)$$

space.

If weights can be subtracted:

$$O(n^{1-1/d} 2^{O(\log^* n)})$$

For reporting:

$$O(n^{1-1/d} 2^{O(\log^* n)} + k)$$

Why Theorem 5.1 Has a Log Factor

At each level, the partition crossing bound loses a constant factor.

With:

$$O(\log \log n)$$

levels, this becomes:

$$(\log n)^{O(1)}$$

To reduce this, use fewer levels and larger one-level partitions. But then the query algorithm can no longer inspect all simplices in a node directly.

Two auxiliary problems are introduced:

- A: report simplices crossed by the boundary of the query simplex.
- B: sum weights of simplices fully contained in the query simplex.

Lemma 6.2: k-Tuple Containment Structure

Input:

Set A of n ordered k-tuples of points, each with semigroup weight.

Query:

Sum weights of k-tuples completely contained in a query simplex.

Output:

Space and preprocessing:

$$O(n(\log n)^{C_k})$$

Query:

$$O(n^{1-1/d} \exp(C'_k \sqrt{\log n}))$$

Construction:

Induct on k.

For k equals 1, use Theorem 5.1.

For k greater than 1:

- Let F be the set of first components.
- Build a partition tree on F.
- At an m-point node choose:

$$r = \frac{m}{s} = \exp(\sqrt{\log m})$$

For each class P_i , store a structure for the remaining $(k-1)$ -tuples.

Query recurrence:

$$T_k(n) \leq O(r) + O(r^{1-1/d})T_k(s) + O(r)T_{k-1}\left(\frac{2n}{r}\right)$$

with:

$$r = \exp(\sqrt{\log n})$$

Using induction:

$$T_{k-1}(n) = O(n^{1-1/d} \exp(C'_{k-1} \sqrt{\log n}))$$

The recurrence gives:

$$T_k(n) = O(n^{1-1/d} \exp(C'_k \sqrt{\log n}))$$

Lemma 6.3: Reporting Segments Crossing a Hyperplane

Input:

n segments in d -space.

Output:

Report all segments crossing a query hyperplane.

Space and preprocessing:

$$O(n(\log n)^{O(1)})$$

Query:

$$O(n^{1-1/d}(\log n)^{O(1)} + k)$$

Construction:

- Build a partition tree on left endpoints.
- At an m -point node choose:

$$s = \lfloor m^{2/3} \rfloor$$

so:

$$r = m^{1/3}$$

- For each class, store a halfspace range reporting structure on corresponding right endpoints.

At an m -point node, there are at most:

$$r = m^{1/3}$$

secondary halfspace queries, each on:

$$O(s) = O(m^{2/3})$$

points.

The additive output terms sum to:

$$O(r^{1-1/d})$$

The overhead terms sum to:

$$O(m^{1/3}(m^{2/3})^{1-1/\lfloor d/2 \rfloor}(\log n)^{O(1)})$$

The paper states this is bounded by:

$$O(m^{1-1/d})$$

for the needed fixed-dimensional comparison.

Depth:

$$O(\log \log n)$$

Final query:

$$O(n^{1-1/d}(\log n)^{O(1)} + k)$$

Theorem 6.1 Final Calculation

Equip each node of the main partition tree with secondary structures for Problems A and B.

At an m -point node, choose class size s so that:

- work in the node is:

$$O(m^{1-1/d})$$

- total secondary space/preprocessing at the node is:

$$O\left(\frac{m}{\log m}\right)$$

Then total space remains:

$$O(n)$$

and preprocessing becomes:

$$O(n^{1+\delta})$$

If node work is:

$$O(m^{1-1/d})$$

and tree depth is D , query time becomes:

$$O(n^{1-1/d} 2^{O(D)})$$

For general semigroup weights, the auxiliary structures allow:

$$s = 2^{C\sqrt{\log m}}$$

Passing one level down reduces:

$$\log m$$

to a constant multiple of:

$$\sqrt{\log m}$$

Thus depth:

$$D = O(\log \log \log n)$$

So:

$$2^{O(D)} = (\log \log n)^{O(1)}$$

and query time:

$$O(n^{1-1/d}(\log \log n)^{O(1)})$$

If weights can be subtracted, or for reporting, Problem B is faster and we may choose:

$$s = (\log m)^{O(1)}$$

Then one level reduces node size from m to:

$$(\log m)^{O(1)}$$

Depth:

$$D = O(\log^* n)$$

Thus query time:

$$O(n^{1-1/d} 2^{O(\log^* n)})$$

For reporting, add output size:

$$O(n^{1-1/d} 2^{O(\log^* n)} + k)$$

Section 7: Dynamization

Theorem 7.1:

The simplex range searching structure of Theorem 5.1 can be maintained under insertions and deletions with:

$$O(\log n)$$

amortized deletion time, and:

$$O(\log^2 n)$$

amortized insertion time.

1.27.1 Deletions

If semigroup weights can be subtracted, update cumulative weights directly along the search path.

Without subtraction:

- Store child weights in a balanced binary tree at each node.
- Updating one child sum costs:

$$O(\log r)$$

at an m-point node.

Since node sizes shrink doubly exponentially down the partition tree, summing over a root-to-leaf path gives:

$$O(\log n)$$

Assign null weights to simulate deletion.

After:

$$\frac{n}{2}$$

deletions, rebuild from scratch to maintain query bounds.

1.27.2 Insertions

The static structure is buildable in:

$$O(n \log n)$$

The range searching problem is decomposable:

$$P = P_1 \cup P_2$$

and the answer on P combines answers on P_1 and P_2 .

Using the Bentley/Overmars dynamization method:

$$O(\log^2 n)$$

amortized insertion time.